Chapter 19: Recovery System

Ensure that the database data is not corrupted or inconsistent in case of system failure

Database Principle and Design

Outline

- Failure Classification
- Recovery scheme
- Stable-Storage Implementation
- Backup and Restore

19.1 Failure Classification

- **System crash**: hardware malfunction, or a bug in the database software or the operating system causes the system to crash. These may result in inconsistent data in the database.
  - handled by recovery scheme
- **Disk failure**: disk failure may destroy all or part of disk storage. These may result in database corruption.
  - handled by stable storage(e.g, RAID)
- **Disaster**: fire, flood, earthquake, war, human operation error. These may also result in database corruption.
  - handled by backup

19.2 Recovery scheme

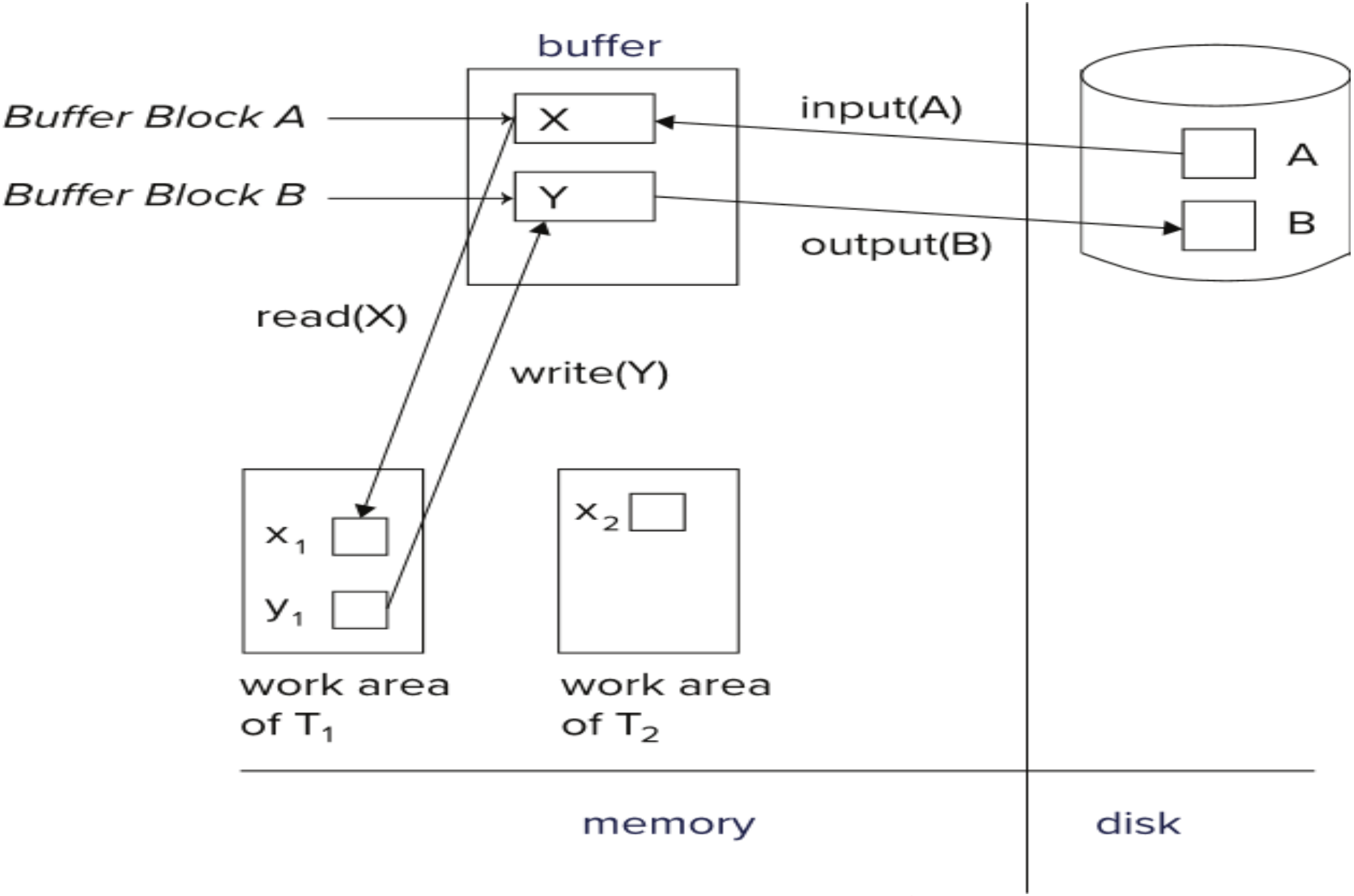
- **Recovery scheme** is an integral part of a database system. It ensures the **atomicity** and **durability** properties of transactions by restoring the database to the consistent state that existed before the failure.
- **Recovery scheme** usually have two parts:
  1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
  2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

19.2.1 Data Access

- Data are transferred between disk and main memory in units of **blocks** (4 to 16 kilobytes typically)
- **Physical blocks** are those blocks residing on the disk.
- Database system reduce the number of disk accesses by keeping as many blocks as possible in main memory.
- **Buffer** is portion of main memory available to store copies of physical blocks. This buffer is shared by all transactions.
- **Buffer blocks** are the blocks residing temporarily in the buffer.
- Block movements between disk and main memory are initiated through the following two operations:
  - **input** (*B*) transfers the physical block *B* to the buffer.
  - **output** (*B*) transfers the buffer block *B* to the disk, and replaces the appropriate physical block there.

Data Access (Cont.)

- Each transaction T_i has its private work-area in which local variables are kept.
- Transaction T_i accesses database using two operations:
  - **read**(X) assigns the value of data item X to the local variable x_i.
    a. If the block B_X on which X resides is not in the buffer, it issues input(B_X).
    b. It assigns to x_i the value of X from the block B_X.
  - **write**(Y) assigns the value of local variable y_i to data item Y in the buffer.
    a. If block B_y on which Y resides is not in the buffer, it issues input(B_Y).
    b. It assigns the value of y_i to Y in the block B_Y



output(B_Y) need not immediately follow **write**(Y). System can perform the output operation when it deems appropriate.

19.3.2 Log-Based Recovery

The most commonly used technique for recovery

Log Records

- A database **log** is a sequence of **log records**. The records keep information about update activities on the database.
- For **log records** to be useful for recovery from system failures, the **log** must be kept on **stable** storage
- Among the types of log records are:
  - < T_i **start** >. Transaction T_i has started.
  - < T_i **commit** >. Transaction T_i has committed.
  - < T_i **abort** >. Transaction T_i has aborted.
  - $<T_i, X, V_1, V_2>$. Transaction T_i has performed a write on data item X. X had value V_1 before the write and has value V_2 after the write.
- The volume of data stored in the log may become unreasonably large. Unlike data in the database, these log information can be deleted at an appropriate time.

Example 19-1

T0	T1	Log	Buffer	Database (A=90, B=60)
		$<T_0$ **start**$>$		
read (A);			A = 90	
A := A – 5;				
write (A);		$<T_0$, A, 90, 85$>$	A = 85	
read (B);			B = 60	
B := B + 5;				
write (B);		$<T_0$, B, 60, 65$>$	B = 65	
commit		$<T_0$ **commit**$>$		A=85, B=65
		$<T_1$ **start**$>$		
	read (B);		B = 65	
	B := B – 10;			
	write (B);	$<T_1$, B, 65, 55$>$	B = 55	
	….	…	…	…
				B = 55
	fail			

Note: The log records of different transactions may be interleaved in the log.

Recovering from Failure

- When recovering after failure:
  - Transaction T_i needs to be redone if the log
    - contains the records $<T_i$ **start**>
    - and contains the record $<T_i$ **commit**> *or* $<T_i$ **abort**>
  - Transaction T_i needs to be undone if the log
    - contains the record $<T_i$ **start**>,
    - but does not contain either the record $<T_i$ **commit**> *or* the record $<T_i$ **abort**>.

Redo and Undo Transactions

- **redo**(T_i) -- sets the value of all data items updated by T_i to the new values, going forward from the first log record for T_i
- **undo**(T_i) -- restores the value of all data items updated by T_i to their old values, going backwards from the last log record for T_i
  - Each time a data item X is restored to its old value V, a special log record $<T_i, X, V>$ is written out, which is called **undo-only** log record,
  - When undo of a transaction is complete, a log record $<T_i$ **abort**$>$ is written out.

Recovering from Failure (Example)

Log	Database (A=90, B=55)
<T_0 **start**>	
<T_0, A, 90, 85>	A=85
<T_0, B, 60, 65>	B=65
<T_0 **commit**>	
<T_1 **start**>	
<T_1, B, 65, 55>	B=65
<T_1, B, 65>	
<T_1 **abort**>	

Example of Database Recovery

Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 1000, 950>	<T_0, A, 1000, 950>	<T_0, A, 1000, 950>
<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 700, 600>	<T_1, C, 700, 600>
		<T_1 commit>

Recovery actions in each case above are:

(a) undo (T_0): B is restored to 2000 and A to 1000, and log records <T_0, B, 2000>, <T_0, A, 1000>, <T_0, **abort**> are written out

(b) redo (T_0) : *A* and *B* are set to 950 and 2050 respectively;

undo (T_1) : C is restored to 700. Log records <T_1, C, 700>, <T_1, **abort**> are written out.

(c) redo (T_0) : A and B are set to 950 and 2050 respectively;

redo (T_1) : *C* is set to 600

Checkpoints

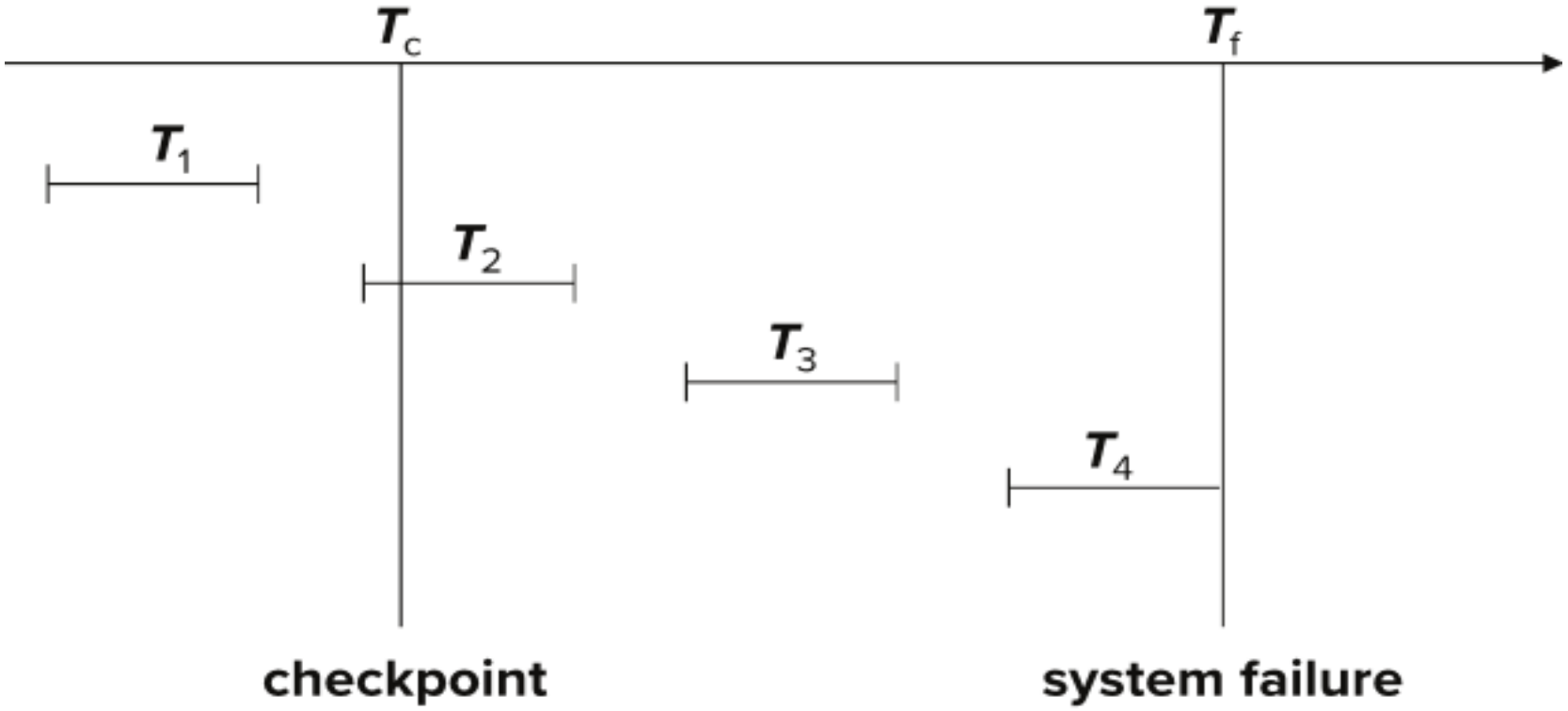
- Redoing/undoing all transactions recorded in the log can be very slow
  - Processing the entire log is time-consuming if the system has run for a long time
  - We might unnecessarily redo transactions which have already output their updates to the database.
- Optimize recovery procedure by periodically performing **checkpointing**

  1. All updates are stopped while doing checkpointing
  2. Output all modified buffer blocks to the disk.
  3. Write a log record < **checkpoint** *L*> where *L* is a list of all transactions active at the time of checkpoint.
- After a system crash has occurred, the system examines the log to find the last < checkpoint L> record. Only transactions that are in *L* or started after the checkpoint need to be redone or undone

Checkpoints(Example 19-1)

T0	T1	Log	Buffer	Database (A=90, B=60)
		$<T_0$ **start**$>$		
read (A);			A = 90	
A := A – 5;				
write (A);		$<T_0$, A, 90, 85$>$	A = 85	
read (B);			B = 60	
B := B + 5;				
write (B);		$<T_0$, B, 60, 65$>$	B = 65	
commit		$<T_0$ **commit**$>$		
		$<T_1$ **start**$>$		
	read (B);			
	B := B – 10;			
	write (B);	$<T_1$, B, 65, 55$>$	B = 55	
	….	…	…	…
		<checkpoint T1>		A=85, B=55

Example of Checkpoints



- T_1 can be ignored (updates already output to disk due to checkpoint)
- T_2 and T_3 redone.
- T_4 undone

19.4 Backup and Restore

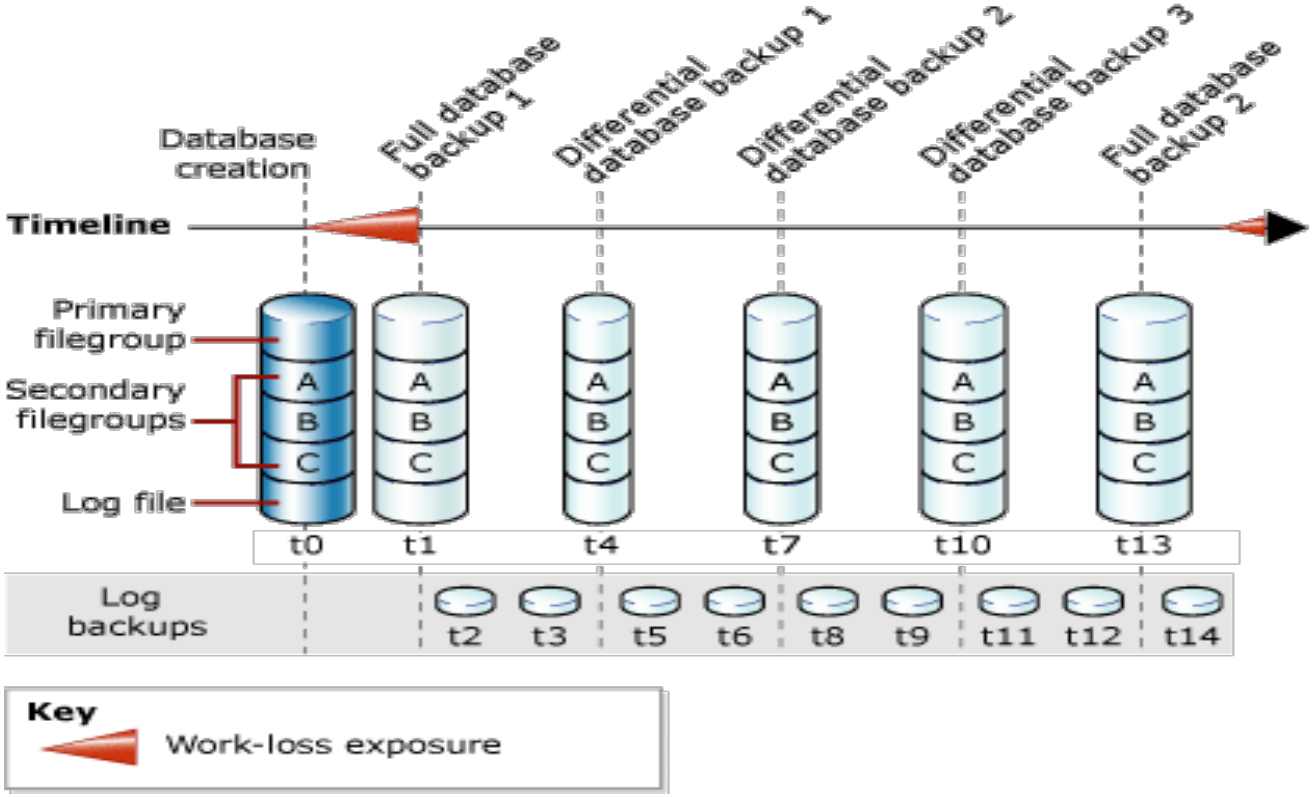
- To survive disaster, the database system needs to use backup and restore technique.
- Database **backup** is to make a copy of data in a database so that the additional copy can be used to restore the original one in case of disaster.
  - The copies generated in the backup should be kept in different locations.
- Database **restore** is to use a backup of a database to recover the database to the state when the backup is made

Types of Backup

- Complete backup
  - Copy all the data in the database
  - Slow and take a lot of disk space when the database is large in scale
- Differential backup
  - Copy data that have been created or changed since the last complete backup
  - Faster and takes less disk space therefore can be done more frequently
- Log backup
  - Backup the logs after last backup
  - Can be done even more frequently

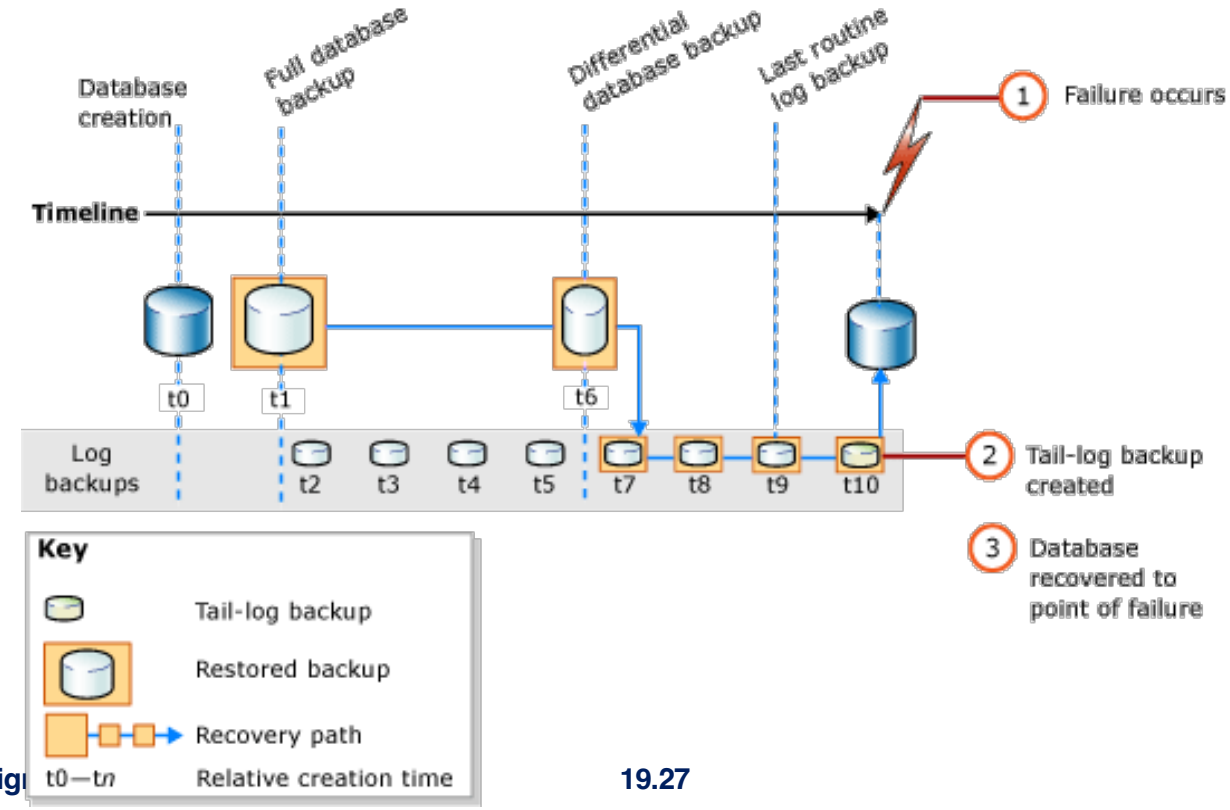
Backup Policy

- Backup policy defines the type and frequency of backups
- Use several types of backups together to achieve efficient and frequent backup
- A typical backup policy:
  - complete backup every week
  - differential backup every day between two complete backups
  - log backup every 2 hours between two non-log backups
- The backup effect of this policy is that at most 2 hours of data changes may be lost.



Data Restore

- Restore the database to the point of failure using database complete backup, differential backup, and log backup
  - If possible, try creating a *tail-log* backup before the failure
  - Restore the most recent complete backup – *complete restore*
  - Restore the most recent differential backup -- *differential restore*
  - Redo all the transactions using log backups taken after the most recent differential backup.– *log restores*



End of Chapter 19